5. Project Development Phase

5.1 Introduction

The development phase converts the design into a functional application. It involves implementing the frontend, backend services, database, authentication, and Google Gemini integration. The components are integrated through REST APIs to form a complete application.

5.2 Development Environment

Technology	Purpose
Python	Backend programming
FastAPI	REST API and backend framework
HTML	Web page structure
CSS	User interface design
JavaScript	Client-side functionality
SQLite / PostgreSQL	Data storage
Google Gemini API	Categorization, budget plans, insights, recommendations, chat
Chart.js	Spending and savings visualization
JWT	User authentication
Visual Studio Code	Development environment
Git / GitHub	Source code management

5.3 Application Development Architecture

User Interface → REST API → FastAPI Backend → Database and User Input → Prompt Builder → Gemini API → Response Processing → User Interface

This separates presentation, business logic, data management, and AI processing, making the application easier to maintain and extend.

5.4 Frontend Development

The frontend is built with HTML, CSS, and JavaScript. It includes login and registration pages, a dashboard, an expense entry form and list, a budget planner page, spending insights, a savings goals page, a recommendation form, a chat interface for the finance assistant, and a history page. The interface is responsive, and loading indicators are displayed while the AI response is being generated. Chart.js is used to display income vs expenses, category breakdown, and savings progress.

5.5 Backend Development

The backend is implemented in Python using FastAPI. It exposes REST endpoints for authentication, transactions, budget generation, insights, goals, recommendations, chat, history, and dashboard data. Pydantic models validate request data, and dependency functions verify JWT tokens on protected routes. Totals, category sums, and savings calculations are computed in

Python so that numeric results are exact and do not depend on the AI model.

5.6 Gemini AI Integration

The backend communicates with the Google Gemini API through its Python SDK. For each feature a dedicated prompt template is used, for example a budget prompt that supplies income, fixed costs, and goals and requests a category-wise plan in a fixed JSON format, or a recommendation prompt that supplies the need and budget limit and requests ranked alternatives with reasons. The backend parses the model's response, handles errors and timeouts, and returns clean results to the frontend. Recent conversation history is included in chat requests so follow-up questions retain context. Each AI answer is accompanied by a short disclaimer that it is general guidance and not professional financial advice.

5.7 Database Development

The database stores users, transactions, budgets, goals, recommendations, and chat history as described in the design phase. SQLAlchemy or direct SQL queries are used for data access, and each record is linked to the user through a user identifier so that users can access only their own data.

5.8 Authentication Implementation

Registration stores a hashed password. Login verifies the credentials and issues a JWT token, which the frontend sends with each request. Protected endpoints reject missing or invalid tokens.

5.9 Challenges and Solutions

Challenge	Solution
Inconsistent AI output format	Strict prompt instructions and JSON parsing with fallback handling
Numerical errors in AI-generated figures	Totals and percentages calculated in the backend; Gemini used only for reasoning and wording
API errors and delays	Try/except handling, retries, and loading messages
Keeping API key and financial data secure	Environment variables, backend-only API calls, per-user data filtering
Maintaining chat context	Sending recent conversation history with each request